

Recitation 5

Timers in Embedded Systems

Table of Contents

1. [Introduction](#)
 2. [Types of Timers](#)
 3. [Timers on the STM32L475](#)
 4. [Mbed OS Timer API Overview](#)
 1. [Timer](#)
 2. [Interrupts and Callbacks](#)
 3. [Timeout](#)
 4. [Ticker](#)
 5. [Quick Reference and References](#)
-

1. Introduction

Almost every embedded application needs timing. Wait 500 ms, blink an LED, read a sensor every second. The question is how you do it.

The most obvious approach is a busy-wait loop:

```
// DO NOT use this in real code
void delay_ms(int ms) {
    volatile int count = ms * 8000; // rough cycle count, not reliable
    while (count > 0) count--;
}
```

The problem with this is that the CPU is stuck doing nothing for the entire delay. You cannot read sensors, respond to inputs, or do any other work. The timing also depends on clock speed and compiler settings, so it is not even accurate.

A hardware timer solves this. It is a counter inside the chip that runs independently of the CPU. You configure it and start it, then the CPU goes and does other work. When the counter reaches its target, it either sets a flag you can check or fires an interrupt to call a function automatically.

2. Types of Timers

Timers can be categorized in two ways: by the clock source that drives them, and by their functional behavior.

By Clock Source

Understanding clock sources matters because they determine accuracy, power consumption, and whether the timer survives sleep modes.

High-Speed External Crystal (HSE) An external crystal oscillator, typically 8 MHz or 16 MHz, connected to dedicated pins on the chip. Crystals are highly accurate because they rely on the mechanical resonance of a quartz element. The main system clock on the STM32L475 is usually derived from HSE via a PLL (Phase-Locked Loop) to reach 80 MHz.

High-Speed Internal RC Oscillator (HSI) A built-in RC (resistor-capacitor) oscillator running at 16 MHz. No external component is needed, but RC oscillators drift with temperature and voltage. Accuracy is typically 1–2%, which is acceptable for many applications but not for precise timing.

Low-Speed External Crystal (LSE) A 32.768 kHz crystal, the standard frequency for real-time clocks. The value 32768 is exactly 2^{15} , which makes it trivial to divide down to 1 Hz using binary counters. The LSE is designed to run continuously, even during deep sleep, at very low power (typically under 1 uA). It drives the RTC and the Low Power Timer on the STM32L475.

Low-Speed Internal RC Oscillator (LSI) A built-in oscillator at approximately 32 kHz. Serves the same purpose as LSE but without an external crystal. Less accurate (can vary by 10–20% across temperature and units) but always available even if no LSE crystal is populated on the board.

Independent Watchdog (IWDG) The IWDG runs exclusively off the LSI oscillator. It is independent from the main clock tree by design — even if the system clock fails, the watchdog keeps running. If the firmware does not periodically "kick" (reset) the watchdog counter, it resets the entire microcontroller. This is a safety mechanism to recover from software hangs or infinite loops.

By Functional Behavior

One-Shot Timer Counts once from zero to a target value, then stops. Used when you need something to happen once after a set time — for example, turning off an LED two seconds after it was turned on.

Periodic Timer Automatically reloads and repeats continuously. Used when you need an action at a fixed rate — for example, sampling a sensor every 100 ms or toggling an LED every 500 ms.

Input Capture The timer records a timestamp when an external signal changes state. Used to measure pulse width, frequency, or the time between two events — for example, measuring the period of a PWM signal coming from a sensor.

PWM Output (Output Compare) The timer drives a digital output high and low at a configured frequency and duty cycle. Used for motor control, servo positioning, LED brightness control, and DAC approximation.

Watchdog Timer A safety timer that resets the system if firmware stops responding. Covered conceptually here but not part of the Mbed Timer API covered in this recitation.

3. Timers on the STM32L475

The timer peripherals themselves live entirely inside the STM32L475VG microcontroller chip (U1 on the board schematic). TIM1 through TIM8, LPTIM1, LPTIM2, SysTick, and the RTC are all silicon peripherals inside the chip. The B-L475E-IOT01A development board adds nothing extra to the timer subsystem.

What the board does contribute are the external clock source components that feed those internal peripherals. The board has specific physical components that determine what clock signals are available to drive the timers. Understanding both layers the board's clock hardware and the chip's timer peripherals is what this section covers.

There are three clock positions on the B-L475E-IOT01A (User Manual UM2153, Section 7.6).

X2 — 32.768 kHz LSE Crystal (fitted) Part number NX3215SA-32.768K, connected to PC14 and PC15. This is the small crystal you can see on the board next to the STM32 chip. It runs continuously even during deep sleep and feeds LPTIM1/2 and the RTC.

X3 — 8 MHz clock from the ST-LINK MCU (fitted) Instead of a dedicated crystal for the main system clock, the board routes the clock output from the ST-LINK chip (STM32F103CBT6) to PH0/PH1 of the main MCU. The STM32L475 takes that 8 MHz signal, runs it through the PLL, and multiplies it up to 80 MHz to run the system clock and all general-purpose timers.

X1 — 8 MHz crystal footprint (not fitted) There is a PCB footprint for a dedicated 8 MHz crystal at position X1, but the component is not populated on this board. X3 covers this role instead. On a custom design you would populate X1.

Note: If you disconnect the ST-LINK in a battery-powered deployment, the STM32L475 loses its HSE source and falls back to the internal HSI at 16 MHz. The LSE crystal X2 keeps running regardless.

Timer Peripherals Inside the STM32L475VG

STM32L4+ Reference Manual (RM0432) — From Page no:1298/2300 in PDF

*### General Purpose Timers — TIM2, TIM3, TIM4, TIM5. STM32L4+ Reference Manual Page no:1298/2300 in PDF

These are the most capable and commonly used timers. Each has:

- A 16-bit or 32-bit up/down counter (TIM2 and TIM5 are 32-bit; TIM3 and TIM4 are 16-bit)
- Four independent capture/compare channels, each of which can be wired to a GPIO pin
- Support for PWM output, input capture, one-pulse mode, and encoder interface
- Clocked from the APB1 bus, derived from the 80 MHz system clock

These timers are what Mbed uses internally for **Ticker** and **Timeout** on this board. Several of their output channels are also routed to the Arduino connector pins (TIM3_CH1 on PB4/D5, TIM3_CH3 on PB0/D3, TIM3_CH4 on PB1/D6, TIM2_CH1 on PA15/D9, TIM2_CH3 on PA2/D10) as visible in the board I/O assignment table.

Advanced Control Timers — TIM1, TIM8

These have everything the general purpose timers have, plus:

- Complementary output channels with configurable dead-time (used for half-bridge motor drivers)
- Break input for emergency shutdown
- Designed specifically for motor control and three-phase PWM generation

Mbed does not expose these advanced features directly. They are accessible via the STM32 HAL if needed.

Basic Timers — TIM6, TIM7

Minimal timers with only a counter and an update event — no I/O channels at all. Used internally for:

- Triggering the DAC at a fixed sample rate
- Generating a time base for software tasks

Mbed uses TIM6 as the `us_ticker` — the microsecond tick source that `Timer`, `Timeout`, and `Ticker` all read from.

Low Power Timer — LPTIM1, LPTIM2

These are fundamentally different from the timers above. They are designed to keep running while the rest of the chip is in Stop or Standby sleep mode. Key properties:

- Clocked from the LSE crystal X2 (32.768 kHz) or the internal LSI oscillator, not the main system clock
- Resolution is limited by the 32 kHz clock — approximately 30 us per tick
- Draw only a few microamps while the rest of the chip sleeps
- Can wake the CPU from sleep when their counter expires

`LowPowerTicker` and `LowPowerTimeout` in Mbed are backed by LPTIM. The LSE crystal X2 on the board is the physical component that makes this work. Without it, LPTIM would fall back to the less accurate internal LSI oscillator.

SysTick

SysTick is not an STM32 peripheral — it is part of the ARM Cortex-M4 core itself. It generates a periodic interrupt (by default every 1 ms in Mbed OS) that drives the RTOS kernel tick. Thread scheduling, `ThisThread::sleep_for`, and timeout management in the RTOS all depend on SysTick. Do not reconfigure SysTick in application code — Mbed OS owns it.

RTC — Real-Time Clock

The RTC is an internal peripheral clocked from the LSE crystal X2. It keeps a calendar (date and time) even during deep sleep and across power cycles if a backup battery is connected to VBAT. `LowPowerTimer` in Mbed uses the RTC for elapsed time measurement that survives sleep.

Hardware to Mbed Mapping

Hardware	Clock Source	Board Component	Mbed Object
TIM6 (us_ticker)	APB1 / 80 MHz	X3 through PLL	<code>Timer</code> , <code>Timeout</code> , <code>Ticker</code>
LPTIM1 / LPTIM2	LSE 32.768 kHz	X2 crystal	<code>LowPowerTicker</code> , <code>LowPowerTimeout</code>
RTC	LSE 32.768 kHz	X2 crystal	<code>LowPowerTimer</code>
SysTick	CPU core clock	X3 through PLL	RTOS kernel (do not touch)

4. Mbed OS Timer API Overview

This recitation covers the three most commonly used timer objects in Mbed OS.

Object	What it does	Interrupt-driven
Timer	Measures elapsed time	No, you check it yourself
Timeout	Calls a function once after a delay	Yes
Ticker	Calls a function repeatedly at a fixed interval	Yes

| **LowPowerTimeout** | One-shot, works during sleep | Yes | | **LowPowerTicker** | Repeating, works during sleep | Yes |

In this recitation we will focus on **Timer**, **Timeout**, and **Ticker**. **If you are interested in lowpower please refer the full version pdf of Recitation 5.** **Timer** is polling-based, meaning you read the elapsed time yourself when you need it. **Timeout** and **Ticker** call a function for you automatically, which means you need to understand interrupts first. That is Section 6.

5. Timer

Timer is a stopwatch. You start it, stop it, and read back how much time has elapsed. It does not fire any interrupt and does not call any function on its own. Everything is driven by your code checking the value.

Under the hood it reads from TIM6 (the us_ticker) at 1 μ s resolution.

When to use it

Use Case	Timer?
Measure how long a function takes to run	Yes
Build a periodic task by checking elapsed time	Yes
Debounce a button	Yes
Check if something has not happened within N seconds	Yes
Trigger a delayed action without blocking main	Use Timeout instead
Blink LED at a precise rate	Use Ticker instead

API

```

Timer t;

t.start();           // start counting
t.stop();            // pause
t.reset();           // reset to zero
t.elapsed_time();    // returns chrono::microseconds
t.read_ms();         // returns int milliseconds (older style, still
works)

```

Polling Pattern

This is how you build a periodic task with `Timer` and no interrupts:

```
#include "mbed.h"

Timer t;
DigitalOut led(LED1);

int main() {
    t.start();
    while (true) {
        if (t.elapsed_time() >= 500ms) {
            led = !led;
            t.reset();
        }
        // other work goes here
    }
}
```

This works fine as long as the other work is quick. If the loop body takes 600 ms, the LED toggles late. This limitation is why `Ticker` exists, which we get to in Section 8.

Exercise 1

Read elapsed time in a loop and print it to serial. Reset the timer every 5 iterations to show reset behavior.

```
#include "mbed.h"

int main() {
    Timer t;
    int iteration = 0;

    printf("Timer demo starting\r\n");
    t.start();

    while (true) {
        iteration++;
        ThisThread::sleep_for(200ms);

        auto elapsed = t.elapsed_time();
        int elapsed_ms = chrono::duration_cast<chrono::milliseconds>(
elapsed).count();
        // chrono::duration_cast converts the elapsed time to milliseconds

        printf("Iteration %d | Elapsed: %d ms\r\n", iteration,
elapsed_ms);

        if (iteration % 5 == 0) {
            printf("Reset timer\r\n");
        }
    }
}
```

```
        t.reset();
    }
}
```

Expected output:

```
Timer demo starting
Iteration 1 | Elapsed: 200 ms
Iteration 2 | Elapsed: 401 ms
Iteration 3 | Elapsed: 602 ms
Iteration 4 | Elapsed: 803 ms
Iteration 5 | Elapsed: 1004 ms
--- Resetting timer ---
Iteration 6 | Elapsed: 201 ms
```

6. Interrupts and Callbacks

Timeout and **Ticker** both call a function automatically when the timer fires. To use them correctly you need a basic understanding of what that means.

What is an Interrupt?

Normally the CPU runs through your code one line at a time. An interrupt is a hardware signal that forces the CPU to stop, run a short special function called an ISR (Interrupt Service Routine), and then return to exactly where it left off.

Normal execution:

```
main() —————>
      line1, line2, line3, line4 ...
```

Timer fires an interrupt:

```
main() ———— PAUSE ———— RESUME ————>
              |         ^
              v         |
            ISR runs ———|
            (short and fast)
```

From main's perspective, a few microseconds just disappeared. It does not know the ISR ran.

What is a Callback?

A callback is a function you write and give to Mbed to call later. When you call **attach()** on a **Ticker**, you are handing Mbed your function and saying "call this every 500 ms." Mbed stores it and calls it from the ISR on schedule.

```
void my_callback() {
    led = !led; // runs inside the ISR
}

ticker.attach(&my_callback, 500ms);
// you never call my_callback() directly
```

ISR Safety Rules

Because the ISR can interrupt your main code at any point, there are strict rules about what you can do inside it.

Not safe inside a Ticker or Timeout callback:

- `printf()` uses a mutex internally and will deadlock or corrupt output
- `malloc()` and `new` are not interrupt-safe
- `ThisThread::sleep_for()` cannot be called from an ISR
- Any Mbed call that uses mutexes, semaphores, or queues

Safe inside an ISR:

- Toggling a `DigitalOut`
- Reading or writing `volatile` variables
- Setting a flag like `volatile bool flag = true`

The rule is: do the minimum inside the ISR, then handle the real work back in main.

The Flag Pattern

This is the standard way to communicate between an ISR and main:

```
volatile bool flag = false; // volatile tells compiler: do not cache this

void on_ticker() {
    flag = true; // safe, simple write
}

int main() {
    ticker.attach(&on_ticker, 500ms);
    while (true) {
        if (flag) {
            flag = false;
            printf("Ticker fired\r\n"); // safe, we are back in main
        }
    }
}
```

7. Timeout

Timeout calls a function once after a specified delay, then stops. Your main program keeps running during the delay. The callback fires automatically when time is up.

When to use it

Use Case	Timeout?
Turn off an LED N seconds after turning it on	Yes
Trigger a retry if no response arrives within 3s	Yes
Detect if a state has been held longer than expected	Yes
Repeat an action at a fixed rate	Use Ticker instead
Measure how long something takes	Use Timer instead

API

```
Timeout t;

t.attach(&my_callback, 2s);    // fire once after 2 seconds
t.attach(&my_callback, 500ms); // fire once after 500 ms
t.detach();                   // cancel before it fires
```

Concept Snippet

```
#include "mbed.h"

DigitalOut led(LED1);
Timeout t;

void turn_off() {
    led = 0; // ISR-safe
}

int main() {
    led = 1;
    t.attach(&turn_off, 2s); // turns off automatically after 2s
    while (true) {
        // main keeps running, no blocking
    }
}
```

Exercise 2

Turn the LED on, schedule it to turn off after 2 seconds using Timeout, and keep printing in main the whole time to show it is not blocked.

```
#include "mbed.h"

DigitalOut led(LED1);
Timeout off_timer;

volatile bool led_off_event = false;

void on_timeout() {
    led = 0;           // turn LED off, ISR-safe
    led_off_event = true; // set flag so main can print
}

int main() {
    printf("Timeout demo starting\r\n");

    led = 1;
    off_timer.attach(&on_timeout, 2s);
    printf("LED on. Will turn off in 2 seconds.\r\n");

    int loop_count = 0;

    while (true) {
        loop_count++;
        ThisThread::sleep_for(200ms);
        printf("Main loop iteration %d\r\n", loop_count);

        if (led_off_event) {
            led_off_event = false;
            printf("LED turned off by Timeout.\r\n");
        }
    }
}
```

Expected output:

```
Timeout demo starting
LED on. Will turn off in 2 seconds.
Main loop iteration 1
Main loop iteration 2
...
Main loop iteration 10
LED turned off by Timeout.
Main loop iteration 11
...
```

8. Ticker

Ticker calls a function at a fixed interval, repeatedly, until you call **detach()**. It is interrupt-driven, so the callback fires on schedule regardless of what main is doing at that moment.

This solves the polling problem from Section 5. Even if the main loop is doing slow work, the LED still toggles at exactly 500 ms.

When to use it

Use Case	Ticker?
Blink an LED at a precise rate	Yes
Sample a sensor at a fixed frequency	Yes, set flag in ISR and read in main
Send a periodic heartbeat	Yes
One-time delayed action	Use Timeout instead

API

```
Ticker t;

t.attach(&my_callback, 500ms); // call every 500 ms
t.attach(&my_callback, 1s);    // call every 1 s
t.detach();                    // stop
```

Concept Snippet

```
#include "mbed.h"

DigitalOut led(LED1);
Ticker blink_ticker;

void toggle_led() {
    led = !led; // ISR-safe
}

int main() {
    blink_ticker.attach(&toggle_led, 500ms);

    while (true) {
        printf("Main loop running\r\n");
        ThisThread::sleep_for(1s); // main sleeps, LED still blinks
    }
}
```

Exercise 3

Blink the LED every 500 ms using Ticker and print a tick counter from main every second.

```
#include "mbed.h"

DigitalOut led(LED1);
Ticker blink_ticker;

volatile int tick_count = 0;

void on_tick() {
    led = !led;    // ISR-safe
    tick_count++;  // ISR-safe, simple integer write
}

int main() {
    printf("Ticker demo starting\r\n");
    printf("LED blinks every 500ms. Main loop runs every 1s.\r\n\r\n");

    blink_ticker.attach(&on_tick, 500ms);

    while (true) {
        ThisThread::sleep_for(1s);
        printf("Main loop | Tick count: %d\r\n", tick_count);
    }
}
```

Expected output:

```
Ticker demo starting
LED blinks every 500ms. Main loop runs every 1s.

Main loop | Tick count: 2
Main loop | Tick count: 4
Main loop | Tick count: 6
Main loop | Tick count: 8
```

The count goes up by 2 each second because the Ticker fires twice per second.

9. Quick Reference

Which Object to Use

Situation	Use This
Measure how long something takes	Timer
Periodic task by polling elapsed time	Timer with <code>elapsed_time()</code> check
Do something once after N ms	Timeout

Situation	Use This
Do something every N ms	<code>Ticker</code>
Forgetting <code>t.start()</code> on a Timer	<code>elapsed_time()</code> always returns zero

API

Timer	– <code>start()</code> , <code>stop()</code> , <code>reset()</code> , <code>elapsed_time()</code>	no ISR, you poll it
Timeout	– <code>attach(callback, delay)</code> , <code>detach()</code>	ISR, fires once
Ticker	– <code>attach(callback, interval)</code> , <code>detach()</code>	ISR, fires repeatedly

References

- Mbed OS 6 Time APIs: <https://os.mbed.com/docs/mbed-os/v6.16/apis/time-apis.html>
- STM32L4 Series Reference Manual (RM0432): In Brightspace
- B-L475E-IOT01A Board User Manual (UM2153): In Brightspace